Mastering Spring Boot MVC & REST API -- Architecture, Patterns, and Enterprise Development Guide

Author: Senior Java Backend Architect

Target Audience: Java backend developers with 3+ years experience aiming for architect-level expertise in Spring Boot web development

Prerequisites: Core Java, Basic SQL, Maven/Gradle fundamentals

Table of Contents

1. Spring Framework Fundamentals
2. Spring Boot Fundamentals
3. Spring MVC Architecture
4. Spring MVC Request Processing Flow
5. Spring Boot MVC Web Application
6. Spring Boot REST API Architecture
7. HTTP Methods and REST Operations
8. Building REST APIs with Spring Boot
9. Request and Response Handling
10. DTO and Data Transfer Layer
11. Validation in Spring Boot REST APIs
12. Global Exception Handling
13. REST API Versioning
14. API Documentation (OpenAPI / Swagger)
15. REST API Security Overview
16. REST API Performance Optimization
17. Complete REST API Project Architecture
18. Best Practices for REST API Design
19. Common REST API Development Mistakes
20. Interview Questions (100+)
21. Hands-on Practice Exercises

Part 1: Spring Framework Fundamentals

1.1 What is the Spring Framework?

Concept Explanation

The **Spring Framework** is a comprehensive application framework for Java that provides infrastructure support for developing enterprise applications. It handles the "plumbing" of your application so you can focus on business logic.

Why Spring Exists

Before Spring, enterprise Java development required:

- Manual object creation and dependency management
- Complex J2EE (EJB) configuration with XML descriptors
- Tight coupling between components -- changing one class broke others
- Boilerplate code for transactions, security, and data access

Spring solved these problems with two fundamental concepts: **Inversion of Control (IoC)** and **Dependency Injection (DI)**.

1.2 Inversion of Control (IoC)

Concept Explanation

Inversion of Control means you do NOT create objects yourself -- the Spring container creates and manages them for you.

The Problem IoC Solves

```
WITHOUT IoC (Traditional):
  OrderService creates its own dependencies:

  class OrderService {
      private OrderRepository repo = new OrderRepository();    // Hardcoded
      private PaymentClient client = new PaymentClient();      // Tight coupling
      private EmailService email = new EmailService();         // Cannot swap
  }

  Problems:
  ✗ Cannot replace PaymentClient with a mock for testing
  ✗ Cannot swap OrderRepository for a different database
  ✗ OrderService KNOWS how to create every dependency
```

```
WITH IoC (Spring):
  Spring creates dependencies and INJECTS them:

  class OrderService {
      private final OrderRepository repo;       // Injected by Spring
      private final PaymentClient client;       // Injected by Spring
      private final EmailService email;         // Injected by Spring

      OrderService(OrderRepository repo, PaymentClient client, EmailService email) {
          this.repo = repo;
          this.client = client;
          this.email = email;
      }
  }

  Benefits:
  ✓ Spring creates OrderRepository, PaymentClient, EmailService
  ✓ Easily swap real implementations with mocks for testing
  ✓ OrderService doesn't know HOW dependencies are created
```

Real-World Analogy

Without IoC: You build your own furniture from raw wood.

With IoC: You order from IKEA -- they build it, deliver it, and assemble it for you. You just USE it.

1.3 Dependency Injection (DI)

Concept Explanation

Dependency Injection is HOW IoC is implemented. Spring "injects" (provides) the objects your class needs, rather than the class creating them.

Three Types of DI

Type	How It Works	Recommended?
Constructor Injection	Dependencies passed via constructor	✅ YES -- immutable, testable
Setter Injection	Dependencies set via setter methods	Optional dependencies only
Field Injection	Dependencies injected directly into fields via `@Autowired`	❌ NO -- untestable, hides dependencies

Why Constructor Injection is Best

```java
// ✅ BEST: Constructor injection
@Service
public class OrderService {
    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;

    // Spring automatically injects dependencies
    public OrderService(OrderRepository orderRepository, PaymentClient paymentClient)
{
        this.orderRepository = orderRepository;
        this.paymentClient = paymentClient;
    }
}

// ❌ AVOID: Field injection
@Service
public class OrderService {
    @Autowired  // Hidden dependency -- cannot see from constructor
    private OrderRepository orderRepository;

    @Autowired  // Cannot create OrderService without Spring
    private PaymentClient paymentClient;
}
```

Why Constructor Injection Wins

Criteria	Constructor	Field
Immutability	`final` fields -- cannot be changed	Mutable -- can be reassigned
Testability	Pass mocks via constructor	Need reflection or Spring context
Visibility	All dependencies visible in constructor	Hidden inside class
Required dependencies	Enforced at compile time	Fails at runtime

1.4 Spring Bean Lifecycle

Concept Explanation

A **Bean** is any object managed by the Spring IoC container. Spring creates, initializes, uses, and destroys beans in a defined lifecycle.

Lifecycle Flow

```
Spring Container Starts
        |
        ▼
  1. Bean Instantiation
     +-- Spring calls constructor (creates object)
        |
        ▼
  2. Dependency Injection
     +-- Spring injects all dependencies
        |
        ▼
  3. @PostConstruct
     +-- Custom initialization (load cache, start connections)
        |
        ▼
  4. Bean Ready for Use
     +-- Application uses the bean normally
        |
        ▼
  5. @PreDestroy
     +-- Cleanup (close connections, flush cache)
        |
        ▼
  6. Bean Destroyed
     +-- Container shuts down
```

Bean Scopes

Scope	Instances	Lifecycle	Use Case
singleton (default)	1 per container	Entire application	Services, repositories
prototype	New instance every request	Per injection point	Stateful beans
request	1 per HTTP request	HTTP request lifecycle	Request-scoped data
session	1 per HTTP session	Session lifecycle	User session data

Code Example

```java
@Service  // Singleton by default -- one instance shared across app
public class CacheService {

    private Map<String, Object> cache;

    @PostConstruct  // Called AFTER dependencies are injected
    public void init() {
        cache = new ConcurrentHashMap<>();
```

```java
        System.out.println("Cache initialized");
    }

    @PreDestroy  // Called BEFORE bean is destroyed
    public void cleanup() {
        cache.clear();
        System.out.println("Cache cleared");
    }
}
```

Part 2: Spring Boot Fundamentals

2.1 What is Spring Boot?

Concept Explanation

Spring Boot is an opinionated extension of the Spring Framework that makes it easy to create stand-alone, production-grade Spring applications. It eliminates most of the configuration boilerplate that traditional Spring requires.

The Problem Spring Boot Solves

```
Traditional Spring Application Setup:
  1. Create Maven project manually
  2. Add 15-20 dependencies individually
  3. Write 200+ lines of XML configuration
  4. Configure web.xml for DispatcherServlet
  5. Configure datasource, transaction manager, entity manager
  6. Set up external Tomcat server
  7. Build WAR file and deploy to server
  Time: 2-4 hours before writing any business code

Spring Boot Application Setup:
  1. Go to start.spring.io
  2. Select dependencies (Web, JPA, MySQL)
  3. Download project
  4. Write business code
  5. Run with: java -jar app.jar
  Time: 5 minutes
```

2.2 Key Spring Boot Features

Auto-Configuration

Spring Boot automatically configures your application based on the dependencies you add:

Dependency Added	Auto-Configuration Applied
`spring-boot-starter-web`	Embedded Tomcat, DispatcherServlet, Jackson JSON
`spring-boot-starter-data-jpa`	EntityManager, TransactionManager, DataSource
`spring-boot-starter-security`	Security filter chain, CSRF protection, login form

How Auto-Configuration Works

```
Spring Boot starts
       |
       ▼
Scans classpath for libraries
       |
       +-- Found: spring-webmvc.jar    -> Configure DispatcherServlet
       +-- Found: hibernate-core.jar   -> Configure EntityManagerFactory
       +-- Found: h2.jar               -> Configure in-memory database
       +-- Found: jackson.jar          -> Configure JSON serialization
       |
       ▼
Applies @ConditionalOnClass, @ConditionalOnMissingBean
       |
       ▼
Creates beans ONLY if not already defined by developer
```

Starter Dependencies

Starters are curated sets of dependencies that work together:

Starter	What It Includes
`spring-boot-starter-web`	Spring MVC, Embedded Tomcat, Jackson, Validation
`spring-boot-starter-data-jpa`	Hibernate, Spring Data JPA, HikariCP connection pool
`spring-boot-starter-security`	Spring Security, authentication, authorization
`spring-boot-starter-test`	JUnit 5, Mockito, AssertJ, MockMvc
`spring-boot-starter-actuator`	Health checks, metrics, monitoring endpoints

Embedded Servers

```
Traditional:                        Spring Boot:
  Write code -> Build WAR ->           Write code -> Build JAR ->
  Install Tomcat -> Deploy WAR ->      java -jar app.jar
  Configure server.xml ->             (Tomcat embedded inside JAR)
  Start Tomcat

  Deployment: Complex                 Deployment: One command
  Scaling: Install Tomcat on each     Scaling: Just copy JAR
```

2.3 Spring Boot Application Structure

```
my-application/
+-- src/main/java/com/company/app/
|   +-- MyApplication.java          <- @SpringBootApplication (entry point)
|   +-- controller/                 <- REST endpoints
|   +-- service/                    <- Business logic
|   +-- repository/                 <- Data access
|   +-- entity/                     <- JPA entities
|   +-- dto/                        <- Data transfer objects
|   +-- config/                     <- Configuration classes
|   +-- exception/                  <- Custom exceptions
+-- src/main/resources/
|   +-- application.yml             <- Application configuration
|   +-- application-dev.yml         <- Dev-specific config
|   +-- application-prod.yml        <- Production config
+-- src/test/java/                  <- Test classes
+-- pom.xml                         <- Maven dependencies
```

@SpringBootApplication -- Three Annotations in One

```java
@SpringBootApplication  // Combines three annotations:
// @Configuration        -> This class provides bean definitions
// @EnableAutoConfiguration -> Enable Spring Boot auto-config
// @ComponentScan        -> Scan this package and sub-packages for beans
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);  // Start the app
    }
}
```

Part 3: Spring MVC Architecture

3.1 What is MVC?

Concept Explanation

MVC (Model-View-Controller) is a design pattern that separates an application into three interconnected components:

Component	Responsibility	Spring Equivalent
Model	Data and business logic	`@Service`, `@Entity`, DTOs
View	UI/presentation layer	Thymeleaf, JSP, JSON response
Controller	Handles requests, connects Model & View	`@Controller`, `@RestController`

Why MVC Exists

Without MVC, a single servlet handles everything -- authentication, business logic, database queries, and HTML generation. This creates a tangled, unmaintainable codebase.

MVC enforces **separation of concerns** -- each component has ONE job.

3.2 Spring MVC Internal Architecture

Architecture Diagram

```
Client (Browser / Mobile / Other Service)
  |
  |  HTTP Request (GET /api/orders/123)
  ▼
+-------------------------------------------------------+
|                    DispatcherServlet                    |
|               (Front Controller Pattern)               |
|                                                        |
|  "I receive ALL requests and route them to the right   |
|   handler. I am the SINGLE entry point."               |
+--------------+-----------------------------------------+
               |
               ▼
+-------------------------+
|      HandlerMapping     |
|                         |
| "I know which controller |
|  handles which URL."    |
```

```
|                            |
| /api/orders/** ->           |
|    OrderController         |
+--------------+-----------+
               |
               ▼
+-------------------------+
|      HandlerAdapter       |
|                           |
| "I know how to invoke     |
|  the controller method."  |
|                           |
| Handles @RequestMapping,  |
| parameter binding, etc.   |
+--------------+-----------+
               |
               ▼
+-------------------------+
|       Controller          |
|    (@RestController)      |
|                           |
| "I process the request    |
|  and return a response."  |
|                           |
| Calls Service -> Repo -> DB|
+--------------+-----------+
               |
               ▼
+-------------------------+
|   HttpMessageConverter    |
|       (Jackson)           |
|                           |
| "I convert Java objects   |
|  to JSON for the response"|
+--------------+-----------+
               |
               ▼
         HTTP Response
    (JSON body + status code)
```

Component Responsibilities

Component	Role	When to Customize
DispatcherServlet	Front Controller -- receives ALL HTTP requests	Rarely (auto-configured)
HandlerMapping	Maps URL patterns to controller methods	When using custom URL strategies
HandlerAdapter	Invokes controller methods, binds parameters	Rarely

Controller	Business logic entry point	Always -- you write these
HttpMessageConverter	Converts Java ↔ JSON/XML	When customizing serialization
ViewResolver	Finds view templates (Thymeleaf, JSP)	For server-rendered pages

---

Part 4: Spring MVC Request Processing Flow

4.1 Step-by-Step Request Lifecycle

Concept Explanation

Every HTTP request in Spring Boot goes through a precise sequence of steps. Understanding this flow is critical for debugging and performance optimization.

Complete Request Flow

```
Step 1: Client sends HTTP request
         |
         |  GET /api/orders/123
         |  Headers: Authorization: Bearer eyJ...
         |
         ▼
Step 2: Servlet Container (Embedded Tomcat)
         |
         |  Creates HttpServletRequest & HttpServletResponse
         |
         ▼
Step 3: Filter Chain
         |
         +-- SecurityFilter (authentication/authorization)
         +-- CorsFilter (cross-origin requests)
         +-- LoggingFilter (request/response logging)
         +-- Custom filters
         |
         ▼
Step 4: DispatcherServlet.doDispatch()
         |
         |  The central routing method
         |
         ▼
Step 5: HandlerMapping.getHandler()
         |
         |  Matches URL to controller method:
         |  GET /api/orders/{id} -> OrderController.getById(Long id)
         |
         ▼
Step 6: HandlerInterceptor.preHandle()
         |
         |  Pre-processing (logging, timing, auth checks)
         |
         ▼
Step 7: HandlerAdapter.handle()
```

```
         |
         +-- Parameter binding:  {id} -> Long id = 123
         +-- @RequestBody -> Jackson deserializes JSON to Java object
         +-- @Valid -> Bean validation runs
         |
         ▼
Step 8: Controller method executes
         |
         |  OrderController.getById(123)
         |    -> OrderService.findById(123)
         |      -> OrderRepository.findById(123)
         |        -> SQL: SELECT * FROM orders WHERE id = 123
         |
         ▼
Step 9: HttpMessageConverter
         |
         |  Jackson converts Order object -> JSON
         |  {"id": 123, "status": "CONFIRMED", "amount": 199.99}
         |
         ▼
Step 10: HandlerInterceptor.afterCompletion()
         |
         |  Post-processing (log response time)
         |
         ▼
Step 11: HTTP Response returned to client
         |
         |  HTTP/1.1 200 OK
         |  Content-Type: application/json
         |  {"id": 123, "status": "CONFIRMED", "amount": 199.99}
```

Key Insight

> *Every request passes through Steps 1-6 and 9-11 automatically. You only write Step 8 (the controller and service logic). Spring handles everything else.*

Part 5: Spring Boot MVC Web Application

5.1 Server-Side Rendering vs REST API

Concept Explanation

Spring MVC supports two types of web applications:

Type	Controller	Returns	Use Case
Server-Side Rendering	`@Controller`	HTML pages (via Thymeleaf)	Admin dashboards, internal tools
REST API	`@RestController`	JSON/XML data	SPAs, mobile apps, microservices

When to Use @Controller (Server-Side Rendering)

- Internal admin panels
- Simple CRUD web applications
- Applications where SEO is critical (server-rendered HTML)
- Rapid prototyping

When to Use @RestController (REST API)

- Mobile app backends
- Single Page Applications (React, Angular)
- Microservices communication
- Third-party API integrations

5.2 @Controller with Thymeleaf

How Server-Side Rendering Works

```
Request Flow (Server-Side Rendering):

  Browser requests: GET /orders

  DispatcherServlet
       |
       ▼
  OrderController
       |
       +-- Calls OrderService.findAll()
       +-- Adds list to Model
       +-- Returns view name: "orders/list"
       |
```

```
      ▼
ViewResolver
      |
      +-- Finds template: templates/orders/list.html
      |
      ▼
Thymeleaf Engine
      |
      +-- Merges data + template -> Complete HTML page
      |
      ▼
Browser receives: Full HTML page (rendered on server)
```

Code Example

```java
@Controller  // Returns VIEW names, not data
@RequestMapping("/orders")
public class OrderViewController {

    private final OrderService orderService;

    public OrderViewController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping
    public String listOrders(Model model) {
        List<Order> orders = orderService.findAll();
        model.addAttribute("orders", orders);    // Add data to model
        return "orders/list";                     // Return VIEW name
    }

    @GetMapping("/{id}")
    public String viewOrder(@PathVariable Long id, Model model) {
        Order order = orderService.findById(id);
        model.addAttribute("order", order);
        return "orders/detail";                   // Return VIEW name
    }

    @GetMapping("/new")
    public String showCreateForm(Model model) {
        model.addAttribute("order", new OrderForm());
        return "orders/create";
    }

    @PostMapping
    public String createOrder(@Valid @ModelAttribute OrderForm form,
                              BindingResult result) {
        if (result.hasErrors()) {
```

```
            return "orders/create";                // Return to form with errors
        }
        orderService.create(form);
        return "redirect:/orders";                 // Redirect after POST
    }
}
```

Part 6: Spring Boot REST API Architecture

6.1 What are REST APIs?

Concept Explanation

REST (Representational State Transfer) is an architectural style for designing networked applications. A REST API exposes **resources** (data objects) that clients can access and manipulate using standard HTTP methods.

Why REST APIs Dominate

Alternative	Problem	REST Advantage
SOAP	Complex XML, WSDL, heavy tooling	Simple JSON, lightweight, human-readable
RPC	Tightly coupled, action-based	Loosely coupled, resource-based
GraphQL	Over-engineering for simple APIs	Simpler for CRUD operations
gRPC	Binary protocol, hard to debug	Text-based HTTP, easy to test with curl

REST Architectural Principles

Principle	What It Means	Why It Matters
Stateless	Each request contains all info needed	Enables horizontal scaling (any server can handle any request)
Resource-based	Everything is a resource with a URL	Uniform, predictable API structure
HTTP methods	Use GET/POST/PUT/DELETE for operations	Leverages existing HTTP semantics
Representation	Resources can be JSON, XML, etc.	Client chooses preferred format
HATEOAS	Response includes links to related resources	Self-discoverable API
Uniform interface	Consistent URL patterns and conventions	Easier to learn and use

6.2 REST API Architecture in Spring Boot

Architecture Diagram

```
+-----------------------------------------------------------------+
|                        CLIENT LAYER                             |
|  Browser  |  Mobile App  |  Other Service  |  Postman/curl     |
+------------------------+----------------------------------------+
                         | HTTP Request (JSON)
                         ▼
+-----------------------------------------------------------------+
|                    CONTROLLER LAYER                             |
|                    @RestController                               |
|                                                                  |
|  Responsibilities:                                               |
|  • Parse HTTP request (path variables, query params, body)       |
|  • Validate input (@Valid)                                       |
|  • Call service layer                                            |
|  • Return HTTP response (status code + body)                     |
|                                                                  |
|  Does NOT contain: Business logic, database queries              |
+------------------------+----------------------------------------+
                         | Method call (Java)
                         ▼
+-----------------------------------------------------------------+
|                     SERVICE LAYER                               |
|                     @Service                                     |
|                                                                  |
|  Responsibilities:                                               |
|  • Business logic and validation rules                           |
|  • Orchestrate multiple repository calls                         |
|  • Transaction management (@Transactional)                       |
|  • DTO ↔ Entity conversion                                      |
|                                                                  |
|  Does NOT contain: HTTP concerns, SQL queries                    |
+------------------------+----------------------------------------+
                         | Method call
                         ▼
+-----------------------------------------------------------------+
|                   REPOSITORY LAYER                              |
|                   @Repository / JpaRepository                   |
|                                                                  |
|  Responsibilities:                                               |
|  • Data access (CRUD operations)                                 |
|  • Custom queries (@Query)                                       |
|  • Spring Data JPA auto-generated queries                        |
|                                                                  |
|  Does NOT contain: Business logic, HTTP concerns                 |
+------------------------+----------------------------------------+
                         | JDBC / JPA
                         ▼
+-----------------------------------------------------------------+
|                       DATABASE                                  |
```

```
|              MySQL / PostgreSQL / MongoDB                        |
+-----------------------------------------------------------------+
```

Why This Layered Architecture?

Without Layers	With Layers
Controller does validation, business logic, SQL	Each layer has ONE responsibility
Changing DB requires changing controller	DB changes only affect repository
Cannot test business logic without HTTP	Service layer testable with Mockito
One developer can break everything	Teams can work on different layers

---

Part 7: HTTP Methods and REST Operations

7.1 Concept Explanation

HTTP Methods Map to CRUD Operations

HTTP Method	CRUD Operation	Purpose	Idempotent?	Safe?
GET	Read	Retrieve a resource	Yes	Yes
POST	Create	Create a new resource	No	No
PUT	Update (full)	Replace entire resource	Yes	No
PATCH	Update (partial)	Modify specific fields	Depends	No
DELETE	Delete	Remove a resource	Yes	No

Key Concepts

Idempotent: Making the same request multiple times produces the same result.

- GET `/orders/1` -> Always returns the same order (idempotent)
- POST `/orders` -> Creates a NEW order each time (NOT idempotent)
- PUT `/orders/1` -> Always sets the same data (idempotent)
- DELETE `/orders/1` -> First call deletes, subsequent calls return 404 (idempotent)

Safe: The request does not modify any data.

- GET is safe -- it only reads
- POST, PUT, DELETE are NOT safe -- they modify data

7.2 REST URL Design

Resource Naming Conventions

```
GOOD URL Design:                        BAD URL Design:
  GET    /api/orders                      GET    /api/getOrders
  GET    /api/orders/123                  GET    /api/getOrderById?id=123
  POST   /api/orders                      POST   /api/createOrder
  PUT    /api/orders/123                  POST   /api/updateOrder
  DELETE /api/orders/123                  GET    /api/deleteOrder?id=123

Rules:
  ✓ Use nouns (orders), not verbs (getOrders)
```

```
  ✓ Use plurals (orders), not singular (order)
  ✓ Use lowercase with hyphens (order-items), not camelCase
  ✓ Use path params for identity (/orders/123)
  ✓ Use query params for filtering (/orders?status=pending)
```

Nested Resources

```
GET    /api/customers/5/orders         -> All orders for customer 5
GET    /api/customers/5/orders/123     -> Order 123 of customer 5
POST   /api/customers/5/orders         -> Create order for customer 5
```

7.3 HTTP Status Codes

Status Code Reference

Code	Meaning	When to Use
200 OK	Request succeeded	Successful GET, PUT, PATCH
201 Created	Resource created	Successful POST
204 No Content	Success, no body	Successful DELETE
400 Bad Request	Invalid input	Validation errors
401 Unauthorized	Not authenticated	Missing or invalid token
403 Forbidden	Not authorized	Authenticated but insufficient permissions
404 Not Found	Resource doesn't exist	GET/PUT/DELETE with invalid ID
409 Conflict	Resource conflict	Duplicate email, concurrent modification
422 Unprocessable Entity	Semantically invalid	Business rule violations
429 Too Many Requests	Rate limited	Client exceeded API rate limit
500 Internal Server Error	Server failure	Unexpected exceptions

Common Mistake

```java
// BAD: Always returning 200 with error in body
@PostMapping("/orders")
public ResponseEntity<Map<String, Object>> createOrder(...) {
    if (invalid) {
        return ResponseEntity.ok(Map.of("error", "Invalid order"));  // 200 with error!
    }
}
```

```
// GOOD: Use proper HTTP status codes
@PostMapping("/orders")
public ResponseEntity<OrderResponse> createOrder(...) {
    if (invalid) {
        return ResponseEntity.badRequest().body(errorResponse);  // 400
    }
    return ResponseEntity.status(HttpStatus.CREATED).body(created);  // 201
}
```

Part 8: Building REST APIs with Spring Boot

8.1 @RestController vs @Controller

Concept Explanation

Spring provides two annotations for handling HTTP requests. Understanding the difference is essential.

Annotation	Returns	Use Case
`@Controller`	View name (HTML template)	Server-side rendered web pages
`@RestController`	Java object -> JSON directly	REST APIs for SPAs, mobile, microservices

What @RestController Does Internally

```
@RestController = @Controller + @ResponseBody

@Controller:
  Method returns "orders/list" -> ViewResolver finds template -> HTML returned

@RestController:
  Method returns Order object -> Jackson converts to JSON -> JSON returned
```

8.2 Mapping Annotations

Annotation Reference

Annotation	HTTP Method	Purpose
`@RequestMapping`	Any (configurable)	Generic mapping -- use for class-level base path
`@GetMapping`	GET	Read/retrieve resources
`@PostMapping`	POST	Create new resources
`@PutMapping`	PUT	Full resource update
`@PatchMapping`	PATCH	Partial resource update
`@DeleteMapping`	DELETE	Remove resources

Why Specific Mappings Exist

```
// OLD way -- error-prone, verbose
@RequestMapping(value = "/orders", method = RequestMethod.GET)
@RequestMapping(value = "/orders", method = RequestMethod.POST)

// NEW way -- clear, concise, self-documenting
@GetMapping("/orders")
@PostMapping("/orders")
```

8.3 Complete CRUD REST Controller

```
@RestController
@RequestMapping("/api/orders")  // Base path for all endpoints in this controller
public class OrderController {

    private final OrderService orderService;

    // Constructor injection -- Spring injects OrderService automatically
    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    // CREATE -- POST /api/orders
    @PostMapping
    public ResponseEntity<OrderResponse> createOrder(
            @Valid @RequestBody OrderRequest request) {

        OrderResponse created = orderService.create(request);
        URI location = URI.create("/api/orders/" + created.getId());

        return ResponseEntity
            .created(location)      // 201 Created + Location header
            .body(created);
    }

    // READ ALL -- GET /api/orders?status=CONFIRMED&page=0&size=20
    @GetMapping
    public ResponseEntity<Page<OrderResponse>> getAllOrders(
            @RequestParam(required = false) String status,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {

        Pageable pageable = PageRequest.of(page, size,
Sort.by("createdAt").descending());
        Page<OrderResponse> orders = orderService.findAll(status, pageable);

        return ResponseEntity.ok(orders);  // 200 OK
```

```java
    }

    // READ ONE -- GET /api/orders/123
    @GetMapping("/{id}")
    public ResponseEntity<OrderResponse> getOrderById(@PathVariable Long id) {
        OrderResponse order = orderService.findById(id);
        return ResponseEntity.ok(order);  // 200 OK
    }

    // UPDATE (Full) -- PUT /api/orders/123
    @PutMapping("/{id}")
    public ResponseEntity<OrderResponse> updateOrder(
            @PathVariable Long id,
            @Valid @RequestBody OrderRequest request) {

        OrderResponse updated = orderService.update(id, request);
        return ResponseEntity.ok(updated);  // 200 OK
    }

    // UPDATE (Partial) -- PATCH /api/orders/123
    @PatchMapping("/{id}")
    public ResponseEntity<OrderResponse> patchOrder(
            @PathVariable Long id,
            @RequestBody Map<String, Object> updates) {

        OrderResponse patched = orderService.patch(id, updates);
        return ResponseEntity.ok(patched);
    }

    // DELETE -- DELETE /api/orders/123
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteOrder(@PathVariable Long id) {
        orderService.delete(id);
        return ResponseEntity.noContent().build();  // 204 No Content
    }
}
```

Code Explanation

Element	Purpose
`@RequestMapping("/api/orders")`	All methods inherit this base path
`@Valid @RequestBody`	Deserializes JSON -> Java object AND runs validation
`@PathVariable Long id`	Extracts `123` from URL `/api/orders/123`
`@RequestParam`	Extracts query parameters `?status=CONFIRMED`
`ResponseEntity.created(location)`	Returns 201 with `Location: /api/orders/456` header

`ResponseEntity.noContent()`	Returns 204 with empty body (for DELETE)

Part 9: Request and Response Handling

9.1 Request Data Extraction

Concept Explanation

Spring provides annotations to extract data from different parts of an HTTP request. Understanding which annotation to use WHERE is critical.

Where Data Lives in HTTP Requests

```
HTTP Request Anatomy:

  POST /api/orders?priority=HIGH HTTP/1.1     <- URL + query params
  Host: api.example.com                       <- Header
  Authorization: Bearer eyJ...                 <- Header
  Content-Type: application/json               <- Header

  {                                            <- Request Body (JSON)
    "productId": 1,
    "quantity": 5,
    "email": "customer@test.com"
  }
```

Annotation Reference

Annotation	Extracts From	Example
`@PathVariable`	URL path	`/orders/{id} -> id = 123`
`@RequestParam`	Query string	`?status=ACTIVE -> status = "ACTIVE"`
`@RequestBody`	Request body (JSON)	JSON -> Java object
`@RequestHeader`	HTTP headers	`Authorization: Bearer ...`
`@CookieValue`	HTTP cookies	Session cookies

9.2 @PathVariable -- Extracting from URL

```java
// Single path variable
@GetMapping("/orders/{id}")
public OrderResponse getById(@PathVariable Long id) {
```

```
    return orderService.findById(id);
}

// Multiple path variables
@GetMapping("/customers/{customerId}/orders/{orderId}")
public OrderResponse getCustomerOrder(
        @PathVariable Long customerId,
        @PathVariable Long orderId) {
    return orderService.findByCustomerAndId(customerId, orderId);
}
```

9.3 @RequestParam -- Query Parameters

```
// Optional params with defaults
@GetMapping("/orders")
public Page<OrderResponse> searchOrders(
        @RequestParam(required = false) String status,              // Optional
        @RequestParam(defaultValue = "createdAt") String sortBy,   // Default value
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {

    return orderService.search(status, sortBy, page, size);
}

// URL: GET /api/orders?status=PENDING&page=2&size=10
```

9.4 @RequestBody -- JSON Deserialization

How JSON -> Java Conversion Works

```
Client sends JSON:
  {
    "productId": 1,
    "quantity": 5,
    "email": "customer@test.com"
  }

Spring uses Jackson ObjectMapper:
  1. Reads Content-Type: application/json
  2. Finds HttpMessageConverter for JSON (MappingJackson2HttpMessageConverter)
  3. Maps JSON fields to Java object fields by name matching
  4. Returns populated Java object

Result:
```

```
OrderRequest request = new OrderRequest();
request.productId = 1L;
request.quantity = 5;
request.email = "customer@test.com";
```

```
@PostMapping("/orders")
public ResponseEntity<OrderResponse> createOrder(
        @Valid @RequestBody OrderRequest request) {  // JSON -> OrderRequest

    OrderResponse response = orderService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

9.5 ResponseEntity -- Controlling HTTP Response

Concept Explanation

`ResponseEntity` gives you full control over the HTTP response -- status code, headers, and body.

```
// ✅ GOOD: Full control with ResponseEntity
@GetMapping("/{id}")
public ResponseEntity<OrderResponse> getById(@PathVariable Long id) {
    OrderResponse order = orderService.findById(id);
    return ResponseEntity
        .ok()                                          // 200 status
        .header("X-Request-Id", UUID.randomUUID().toString())  // Custom header
        .body(order);                                  // Response body
}

// Common ResponseEntity patterns:
ResponseEntity.ok(body)                     // 200 + body
ResponseEntity.created(uri).body(body)      // 201 + Location header + body
ResponseEntity.noContent().build()          // 204 + no body
ResponseEntity.badRequest().body(error)     // 400 + error body
ResponseEntity.notFound().build()           // 404 + no body
ResponseEntity.status(HttpStatus.CONFLICT).body(error)  // 409
```

Part 10: DTO and Data Transfer Layer

10.1 What is a DTO?

Concept Explanation

A **DTO (Data Transfer Object)** is a plain Java object used to transfer data between layers. It separates your API contract from your database schema.

Why DTOs Exist

```
WITHOUT DTOs (Expose entity directly):

  @Entity
  class User {
      private Long id;
      private String name;
      private String email;
      private String passwordHash;     <- LEAKED to API response!
      private String ssn;              <- LEAKED to API response!
      private LocalDateTime deletedAt; <- Internal audit field exposed!
      @OneToMany
      private List<Order> orders;      <- Lazy loading error / N+1 queries!
  }

WITH DTOs (Controlled response):

  class UserResponse {
      private Long id;
      private String name;
      private String email;
      // No password, no SSN, no internal fields
  }
```

Entity vs DTO

Aspect	Entity	DTO
Purpose	Represents database table	Represents API request/response
Annotations	`@Entity`, `@Table`, `@Column`	None (POJO) or `@Valid`
Contains everything?	Yes -- all columns	No -- only what client needs
Security	May have sensitive fields	Sensitive fields excluded

Coupling	Tied to database schema	Tied to API contract

10.2 DTO Pattern in Practice

Request DTO (Client -> Server)

```
// What the client SENDS to create an order
public class OrderRequest {
    @NotNull(message = "Product ID is required")
    private Long productId;

    @Min(value = 1, message = "Quantity must be at least 1")
    private int quantity;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String customerEmail;

    // Getters, Setters, Constructors
}
```

Response DTO (Server -> Client)

```
// What the server RETURNS to the client
public class OrderResponse {
    private Long id;
    private String productName;        // Resolved from Product entity
    private int quantity;
    private BigDecimal totalAmount;    // Calculated field
    private String status;
    private LocalDateTime createdAt;

    // No internal fields (audit columns, foreign keys, etc.)
}
```

Entity (Database representation)

```
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```java
    @ManyToOne(fetch = FetchType.LAZY)
    private Product product;           // FK relationship

    private int quantity;
    private BigDecimal totalAmount;
    private String status;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Column(name = "created_by")
    private String createdBy;          // Audit field -- NOT in response
}
```

10.3 DTO Mapping Approaches

Manual Mapping (Simple, Explicit)

```java
@Service
public class OrderService {

    public OrderResponse toResponse(Order order) {
        OrderResponse response = new OrderResponse();
        response.setId(order.getId());
        response.setProductName(order.getProduct().getName());
        response.setQuantity(order.getQuantity());
        response.setTotalAmount(order.getTotalAmount());
        response.setStatus(order.getStatus());
        response.setCreatedAt(order.getCreatedAt());
        return response;
    }

    public Order toEntity(OrderRequest request) {
        Order order = new Order();
        order.setQuantity(request.getQuantity());
        order.setCustomerEmail(request.getCustomerEmail());
        return order;
    }
}
```

MapStruct (Compile-time code generation)

```java
@Mapper(componentModel = "spring")
public interface OrderMapper {

    @Mapping(source = "product.name", target = "productName")
    OrderResponse toResponse(Order order);

    Order toEntity(OrderRequest request);

    List<OrderResponse> toResponseList(List<Order> orders);
}

// Usage:
@Service
public class OrderService {
    private final OrderMapper mapper;  // Injected by Spring

    public OrderResponse findById(Long id) {
        Order order = orderRepository.findById(id)
            .orElseThrow(() -> new OrderNotFoundException(id));
        return mapper.toResponse(order);  // One-liner mapping
    }
}
```

Part 11: Validation in Spring Boot REST APIs

11.1 Why Input Validation?

Concept Explanation

Input validation ensures that data received from clients is correct, complete, and safe BEFORE it reaches your business logic or database.

What Happens Without Validation

```
Without validation:
  POST /api/users
  {
    "name": "",              <- Empty name saved to DB
    "email": "not-an-email", <- Invalid email saved
    "age": -5                <- Negative age saved
  }

  Result: Corrupted database, broken business logic, security vulnerabilities

With validation:
  POST /api/users -> 400 Bad Request
  {
    "errors": [
      "Name must not be blank",
      "Email must be a valid email address",
      "Age must be at least 0"
    ]
  }
```

11.2 Validation Annotations

Annotation	Purpose	Example
`@NotNull`	Field cannot be null	`@NotNull Long id`
`@NotBlank`	String cannot be null, empty, or whitespace	`@NotBlank String name`
`@NotEmpty`	Collection/string cannot be null or empty	`@NotEmpty List items`
`@Size`	String/collection size constraints	`@Size(min=2, max=50) String name`
`@Min` / `@Max`	Numeric range	`@Min(1) int quantity`

`@Email`	Valid email format	`@Email String email`
`@Pattern`	Regex pattern match	`@Pattern(regexp="[A-Z]{2}") String code`
`@Positive`	Must be > 0	`@Positive BigDecimal amount`
`@Past` / `@Future`	Date constraints	`@Past LocalDate birthDate`

11.3 Validation in Action

```
// Request DTO with validation rules
public class CreateUserRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be 2-50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be a valid email address")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, max = 100, message = "Password must be 8-100 characters")
    @Pattern(regexp = ".*[A-Z].*", message = "Password must contain an uppercase
letter")
    @Pattern(regexp = ".*[0-9].*", message = "Password must contain a digit")
    private String password;

    @Min(value = 18, message = "Must be at least 18 years old")
    @Max(value = 150, message = "Age must be less than 150")
    private int age;

    // Getters, Setters
}

// Controller -- @Valid triggers validation
@PostMapping("/users")
public ResponseEntity<UserResponse> createUser(
        @Valid @RequestBody CreateUserRequest request) {  // @Valid activates checks

    UserResponse user = userService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(user);
}
```

How @Valid Works Internally

```
POST /api/users arrives
       |
       ▼
Jackson deserializes JSON -> CreateUserRequest
       |
       ▼
@Valid triggers Hibernate Validator
       |
       +-- @NotBlank("name") -> Check name is not blank
       +-- @Email("email")   -> Check email format
       +-- @Size("password") -> Check password length
       +-- @Min("age")       -> Check age >= 18
       |
       +-- All valid -> Controller method executes normally
       +-- Any invalid -> MethodArgumentNotValidException thrown -> 400 Bad Request
```

11.4 Custom Validators

```java
// Custom annotation
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueEmailValidator.class)
public @interface UniqueEmail {
    String message() default "Email already exists";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// Validator implementation
public class UniqueEmailValidator implements ConstraintValidator<UniqueEmail, String>
{

    private final UserRepository userRepository;

    public UniqueEmailValidator(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public boolean isValid(String email, ConstraintValidatorContext context) {
        return email != null && !userRepository.existsByEmail(email);
    }
}

// Usage:
public class CreateUserRequest {
    @UniqueEmail
```

```
    private String email;
}
```

Part 12: Global Exception Handling

12.1 Why Global Exception Handling?

Concept Explanation

Without centralized exception handling, every controller method needs its own try-catch block. This leads to duplicated error handling code and inconsistent error responses.

Problem Without Global Handling

```
// BAD: Every controller method handles exceptions individually
@GetMapping("/{id}")
public ResponseEntity<?> getOrder(@PathVariable Long id) {
    try {
        return ResponseEntity.ok(orderService.findById(id));
    } catch (OrderNotFoundException e) {
        return ResponseEntity.notFound().build();
    } catch (UnauthorizedException e) {
        return ResponseEntity.status(401).body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Internal error");
    }
}
// Repeated in EVERY controller method!
```

12.2 @ControllerAdvice + @ExceptionHandler

How It Works

```
Exception thrown anywhere in application
         |
         ▼
Spring searches for @ControllerAdvice class
         |
         ▼
Finds matching @ExceptionHandler method
         |
         ▼
Handler builds proper error response
         |
```

```
        ▼
HTTP response returned to client
```

Implementation

```java
// Standardized error response structure
public class ApiError {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private List<FieldError> fieldErrors;  // For validation errors

    public record FieldError(String field, String message) {}
}

// Global exception handler -- handles ALL exceptions centrally
@RestControllerAdvice  // @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    // 404 -- Resource not found
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(
            ResourceNotFoundException ex, HttpServletRequest request) {

        ApiError error = new ApiError();
        error.setTimestamp(LocalDateTime.now());
        error.setStatus(404);
        error.setError("Not Found");
        error.setMessage(ex.getMessage());
        error.setPath(request.getRequestURI());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    // 400 -- Validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest request) {

        List<ApiError.FieldError> fieldErrors = ex.getBindingResult()
            .getFieldErrors().stream()
            .map(err -> new ApiError.FieldError(err.getField(),
err.getDefaultMessage()))
            .toList();

        ApiError error = new ApiError();
        error.setTimestamp(LocalDateTime.now());
```

```
        error.setStatus(400);
        error.setError("Validation Failed");
        error.setMessage("Input validation failed");
        error.setPath(request.getRequestURI());
        error.setFieldErrors(fieldErrors);

        return ResponseEntity.badRequest().body(error);
    }

    // 409 -- Conflict (duplicate resource)
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ApiError> handleDuplicate(
            DuplicateResourceException ex, HttpServletRequest request) {

        ApiError error = new ApiError();
        error.setTimestamp(LocalDateTime.now());
        error.setStatus(409);
        error.setError("Conflict");
        error.setMessage(ex.getMessage());
        error.setPath(request.getRequestURI());

        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    // 500 -- Catch-all for unexpected errors
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleGeneric(
            Exception ex, HttpServletRequest request) {

        ApiError error = new ApiError();
        error.setTimestamp(LocalDateTime.now());
        error.setStatus(500);
        error.setError("Internal Server Error");
        error.setMessage("An unexpected error occurred");  // Don't expose internals
        error.setPath(request.getRequestURI());

        // Log the actual error for debugging
        log.error("Unexpected error on {}: {}", request.getRequestURI(),
ex.getMessage(), ex);

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

Custom Exception Classes

```
// Base exception for all "not found" errors
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, Long id) {
```

```
        super(resource + " not found with id: " + id);
    }
}

// Specific exceptions
public class OrderNotFoundException extends ResourceNotFoundException {
    public OrderNotFoundException(Long id) {
        super("Order", id);
    }
}

public class DuplicateResourceException extends RuntimeException {
    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

Part 13: REST API Versioning

13.1 Why Version APIs?

Concept Explanation

API versioning allows you to make **breaking changes** to your API without breaking existing clients. When your API evolves, old clients should continue working while new clients use the latest version.

When Versioning is Needed

- Removing a field from a response
- Renaming a field
- Changing the type of a field
- Restructuring the response format

13.2 Versioning Strategies

Strategy 1: URI Versioning (Most Common)

```
// Version in the URL path
@RestController
@RequestMapping("/api/v1/orders")
public class OrderControllerV1 {
    @GetMapping("/{id}")
    public OrderResponseV1 getById(@PathVariable Long id) { ... }
}

@RestController
@RequestMapping("/api/v2/orders")
public class OrderControllerV2 {
    @GetMapping("/{id}")
    public OrderResponseV2 getById(@PathVariable Long id) { ... }
}

// Client usage:
// GET /api/v1/orders/123  -> Old format
// GET /api/v2/orders/123  -> New format
```

Pros	Cons
Simple and obvious	URL changes for every version

Easy to test with browser/curl	Can lead to code duplication
Clear API documentation	Not truly RESTful (URL represents version, not resource)

Strategy 2: Header Versioning

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @GetMapping(value = "/{id}", headers = "X-API-Version=1")
    public OrderResponseV1 getByIdV1(@PathVariable Long id) { ... }

    @GetMapping(value = "/{id}", headers = "X-API-Version=2")
    public OrderResponseV2 getByIdV2(@PathVariable Long id) { ... }
}

// Client usage:
// GET /api/orders/123  Header: X-API-Version: 1
// GET /api/orders/123  Header: X-API-Version: 2
```

Strategy 3: Media Type Versioning (Content Negotiation)

```
@GetMapping(value = "/{id}", produces = "application/vnd.company.v1+json")
public OrderResponseV1 getByIdV1(@PathVariable Long id) { ... }

@GetMapping(value = "/{id}", produces = "application/vnd.company.v2+json")
public OrderResponseV2 getByIdV2(@PathVariable Long id) { ... }

// Client usage:
// GET /api/orders/123  Accept: application/vnd.company.v1+json
```

Recommendation

Strategy	When to Use
URI versioning	Public APIs, simplicity, most teams
Header versioning	Internal APIs, clean URLs needed
Media type versioning	Large enterprise APIs with strict content negotiation

---

Part 14: API Documentation (OpenAPI / Swagger)

14.1 Why API Documentation?

Concept Explanation

API documentation tells consumers (frontend developers, mobile developers, third-party integrators) HOW to use your API -- what endpoints exist, what parameters they expect, and what responses they return.

Without Documentation

```
Frontend developer: "What's the API endpoint for creating an order?"
Backend developer: "POST /api/orders with... let me check my code... hold on..."
-> Wastes time, error-prone, knowledge locked in code
```

14.2 OpenAPI + SpringDoc

What is OpenAPI?

OpenAPI (formerly Swagger) is a specification for describing REST APIs. **SpringDoc** automatically generates OpenAPI documentation from your Spring Boot code.

Setup

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>
```

After adding this dependency:

- `http://localhost:8080/swagger-ui.html` -- Interactive UI
- `http://localhost:8080/v3/api-docs` -- Raw OpenAPI JSON

Enhancing Documentation with Annotations

```
@Tag(name = "Orders", description = "Order management endpoints")
@RestController
```

```java
@RequestMapping("/api/v1/orders")
public class OrderController {

    @Operation(
        summary = "Create a new order",
        description = "Creates an order and processes payment",
        responses = {
            @ApiResponse(responseCode = "201", description = "Order created
successfully"),
            @ApiResponse(responseCode = "400", description = "Invalid order data"),
            @ApiResponse(responseCode = "409", description = "Duplicate order")
        }
    )
    @PostMapping
    public ResponseEntity<OrderResponse> createOrder(
            @Valid @RequestBody OrderRequest request) {
        // ...
    }

    @Operation(summary = "Get order by ID")
    @ApiResponse(responseCode = "200", description = "Order found")
    @ApiResponse(responseCode = "404", description = "Order not found")
    @GetMapping("/{id}")
    public ResponseEntity<OrderResponse> getById(
            @Parameter(description = "Order ID", example = "123")
            @PathVariable Long id) {
        // ...
    }
}
```

Configuration

```java
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI openAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("E-Commerce Order Service API")
                .description("REST API for managing orders, customers, and payments")
                .version("1.0")
                .contact(new Contact()
                    .name("Backend Team")
                    .email("backend@company.com")))
            .addSecurityItem(new SecurityRequirement().addList("Bearer"))
            .components(new Components()
                .addSecuritySchemes("Bearer", new SecurityScheme()
                    .type(SecurityScheme.Type.HTTP)
```

```
                        .scheme("bearer")
                        .bearerFormat("JWT")));
    }
}
```

Part 15: REST API Security Overview

15.1 Why API Security?

Concept Explanation

API security protects your backend from unauthorized access, data theft, and malicious attacks. Without security, anyone can read, modify, or delete your data.

Threat Model

```
Common API Security Threats:

  1. Unauthenticated access     -> Anyone can access private data
  2. Unauthorized actions        -> User accesses admin endpoints
  3. SQL injection              -> Malicious SQL in request params
  4. Data exposure              -> Passwords/SSNs in API responses
  5. Rate limiting bypass       -> DDoS attack via unlimited requests
  6. Token theft                -> Stolen JWT used to impersonate user
```

15.2 Authentication vs Authorization

Concept	Question	Example
Authentication	WHO are you?	Login with username/password -> receive JWT
Authorization	WHAT can you do?	Admin can delete users, regular user cannot

15.3 JWT Authentication Flow

```
JWT Authentication Architecture:

  Step 1: Login
    Client -> POST /api/auth/login { email, password }
    Server -> Validates credentials
    Server -> Creates JWT token
    Server -> Returns { "token": "eyJhbG..." }

  Step 2: Authenticated Request
    Client -> GET /api/orders
              Header: Authorization: Bearer eyJhbG...
    Server -> JwtAuthFilter extracts token
```

```
    Server -> Validates signature and expiry
    Server -> Sets SecurityContext
    Server -> Controller method executes

  Step 3: Token Rejected
    Client -> GET /api/orders
              Header: Authorization: Bearer expired_token
    Server -> JwtAuthFilter validates -> INVALID
    Server -> Returns 401 Unauthorized
```

15.4 Spring Security Configuration

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())                  // Disable CSRF for APIs
            .sessionManagement(session ->
                session.sessionCreationPolicy(STATELESS))  // No sessions -- JWT only
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()           // Public
                .requestMatchers("/api/admin/**").hasRole("ADMIN")     // Admin only
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()  //
Public reads
                .anyRequest().authenticated())             // Everything else needs
auth
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

15.5 OAuth2 Overview

```
OAuth2 Flow (Simplified):

  User -> "Login with Google" button
    |
    ▼
  Redirect to Google login page
    |
    ▼
```

```
  User enters Google credentials
    |
    ▼
  Google redirects back with authorization code
    |
    ▼
  Server exchanges code for access token
    |
    ▼
  Server creates local session / JWT
    |
    ▼
  User is authenticated

Use Cases:
  • Social login (Google, GitHub, Facebook)
  • Enterprise SSO (Okta, Azure AD)
  • API gateway authentication
```

Part 16: REST API Performance Optimization

16.1 Pagination

Why Pagination?

Without pagination, `GET /api/orders` returns ALL 10 million orders in one response -- out-of-memory crash.

```
// Paginated endpoint
@GetMapping
public Page<OrderResponse> getOrders(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(defaultValue = "createdAt") String sortBy,
        @RequestParam(defaultValue = "desc") String direction) {

    Sort sort = direction.equalsIgnoreCase("asc")
        ? Sort.by(sortBy).ascending()
        : Sort.by(sortBy).descending();

    Pageable pageable = PageRequest.of(page, size, sort);
    return orderService.findAll(pageable);
}

// Response includes pagination metadata:
// {
//   "content": [...],
//   "totalElements": 10000,
//   "totalPages": 500,
//   "number": 0,
//   "size": 20,
//   "first": true,
//   "last": false
// }
```

16.2 Caching

Spring Cache Abstraction

```
@Service
public class ProductService {
```

```java
    @Cacheable(value = "products", key = "#id")  // Cache result
    public ProductResponse findById(Long id) {
        return productRepository.findById(id)
            .map(this::toResponse)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }

    @CacheEvict(value = "products", key = "#id")  // Clear cache on update
    public ProductResponse update(Long id, ProductRequest request) {
        // ...
    }

    @CacheEvict(value = "products", allEntries = true)  // Clear all
    public void clearCache() {
        // Cache cleared
    }
}
```

Configuration with Redis

```yaml
# application.yml
spring:
  cache:
    type: redis
    redis:
      time-to-live: 3600000   # 1 hour TTL
  redis:
    host: localhost
    port: 6379
```

16.3 Response Compression

```yaml
# application.yml
server:
  compression:
    enabled: true
    mime-types: application/json,application/xml,text/html
    min-response-size: 1024    # Compress responses > 1KB
```

16.4 Rate Limiting

```java
// Using Bucket4j with Spring Boot
@RestController
@RequestMapping("/api")
public class RateLimitedController {

    private final Bucket bucket = Bucket.builder()
        .addLimit(Bandwidth.classic(100, Refill.intervally(100,
Duration.ofMinutes(1))))
        .build();  // 100 requests per minute

    @GetMapping("/resource")
    public ResponseEntity<?> getResource() {
        if (bucket.tryConsume(1)) {
            return ResponseEntity.ok(data);
        }
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .header("Retry-After", "60")
            .body("Rate limit exceeded. Try again later.");
    }
}
```

Part 17: Complete REST API Project Architecture

17.1 E-Commerce Order Service Architecture

Project Structure

```
order-service/
+-- src/main/java/com/company/orderservice/
|   |
|   +-- OrderServiceApplication.java         <- Entry point
|   |
|   +-- config/                              <- Configuration
|   |   +-- SecurityConfig.java              <- Security rules
|   |   +-- OpenApiConfig.java               <- Swagger setup
|   |   +-- CacheConfig.java                 <- Redis cache config
|   |   +-- WebConfig.java                   <- CORS, interceptors
|   |
|   +-- controller/                          <- REST Endpoints
|   |   +-- OrderController.java             <- /api/v1/orders
|   |   +-- ProductController.java           <- /api/v1/products
|   |   +-- CustomerController.java          <- /api/v1/customers
|   |
|   +-- service/                             <- Business Logic
|   |   +-- OrderService.java
|   |   +-- ProductService.java
|   |   +-- CustomerService.java
|   |   +-- PaymentService.java              <- External payment calls
|   |
|   +-- repository/                          <- Data Access
|   |   +-- OrderRepository.java
|   |   +-- ProductRepository.java
|   |   +-- CustomerRepository.java
|   |
|   +-- entity/                              <- JPA Entities
|   |   +-- Order.java
|   |   +-- OrderItem.java
|   |   +-- Product.java
|   |   +-- Customer.java
|   |
|   +-- dto/                                 <- Data Transfer Objects
|   |   +-- request/
|   |   |   +-- OrderRequest.java
|   |   |   +-- ProductRequest.java
|   |   +-- response/
|   |       +-- OrderResponse.java
|   |       +-- ProductResponse.java
|   |       +-- PagedResponse.java
```

```
|   |
|   +-- mapper/                              <- DTO ↔ Entity Mapping
|   |   +-- OrderMapper.java
|   |   +-- ProductMapper.java
|   |
|   +-- exception/                           <- Exception Handling
|   |   +-- GlobalExceptionHandler.java
|   |   +-- ResourceNotFoundException.java
|   |   +-- DuplicateResourceException.java
|   |   +-- ApiError.java
|   |
|   +-- security/                            <- Authentication
|   |   +-- JwtAuthFilter.java
|   |   +-- JwtTokenProvider.java
|   |   +-- UserDetailsServiceImpl.java
|   |
|   +-- util/                                <- Utilities
|       +-- Constants.java
|       +-- DateUtils.java
|
+-- src/main/resources/
|   +-- application.yml
|   +-- application-dev.yml
|   +-- application-prod.yml
|
+-- src/test/java/                           <- Tests mirror main
|   +-- com/company/orderservice/
|       +-- controller/
|       +-- service/
|       +-- repository/
|       +-- integration/
|
+-- pom.xml
```

17.2 Layer Responsibilities

Layer	Package	Responsibility	Should NOT Do
Controller	`controller/`	Parse HTTP, validate, return response	Business logic, SQL queries
Service	`service/`	Business logic, orchestration, transactions	HTTP handling, SQL
Repository	`repository/`	Data access, queries	Business logic, HTTP
Entity	`entity/`	Database table mapping	Business logic
DTO	`dto/`	API contract (request/response)	Database mapping
Exception	`exception/`	Centralized error handling	Business logic

Config	`config/`	Bean configuration, security	Business logic

Part 18: Best Practices for REST API Design

18.1 URL Design Rules

Rule	Good Example	Bad Example
Use nouns, not verbs	`GET /api/orders`	`GET /api/getOrders`
Use plurals	`/api/orders`	`/api/order`
Use hyphens for multi-word	`/api/order-items`	`/api/orderItems`
Use path params for identity	`/api/orders/123`	`/api/orders?id=123`
Use query params for filtering	`/api/orders?status=PENDING`	`/api/orders/status/PENDING`
Nest related resources	`/api/customers/5/orders`	`/api/getOrdersByCustomer?id=5`

18.2 Consistent Response Format

```
// GOOD: Every API response follows the same structure
// Success response:
{
    "status": "success",
    "data": { ... },
    "timestamp": "2024-01-15T10:30:00Z"
}

// Error response:
{
    "status": "error",
    "error": {
        "code": 404,
        "message": "Order not found with id: 123",
        "path": "/api/orders/123"
    },
    "timestamp": "2024-01-15T10:30:00Z"
}
```

18.3 HTTP Status Code Rules

Scenario	Status Code	Never Use

GET success	200 OK	201, 204
POST success	201 Created	200
PUT/PATCH success	200 OK	201
DELETE success	204 No Content	200 with body
Validation failure	400 Bad Request	200 with error message
Not authenticated	401 Unauthorized	403
Not authorized	403 Forbidden	401
Resource not found	404 Not Found	200 with null
Server error	500 Internal Server Error	200 with error

18.4 Naming Conventions

```
Controller:     OrderController           (noun + Controller)
Service:        OrderService              (noun + Service)
Repository:     OrderRepository           (noun + Repository)
Entity:         Order                     (singular noun)
Request DTO:    CreateOrderRequest        (verb + noun + Request)
                UpdateOrderRequest
Response DTO:   OrderResponse             (noun + Response)
Exception:      OrderNotFoundException    (noun + NotFound + Exception)
```

Part 19: Common REST API Development Mistakes

19.1 Using Wrong HTTP Methods

```
// BAD: Using GET for creating data
@GetMapping("/createOrder")
public Order create(@RequestParam String product) { ... }

// BAD: Using POST for reading data
@PostMapping("/searchOrders")
public List<Order> search(@RequestBody SearchRequest request) { ... }

// GOOD: Use correct HTTP methods
@PostMapping("/orders")
public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest request) { ...
}

@GetMapping("/orders")
public Page<OrderResponse> search(@RequestParam String status) { ... }
```

19.2 Returning Wrong Status Codes

```
// BAD: Always returning 200
@PostMapping("/orders")
public ResponseEntity<Map<String, Object>> create(...) {
    try {
        Order order = orderService.create(request);
        return ResponseEntity.ok(Map.of("order", order));  // 200 for creation!
    } catch (Exception e) {
        return ResponseEntity.ok(Map.of("error", e.getMessage()));  // 200 for error!
    }
}

// GOOD: Proper status codes
@PostMapping("/orders")
public ResponseEntity<OrderResponse> create(@Valid @RequestBody OrderRequest request)
{
    OrderResponse order = orderService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(order);  // 201
}
// Errors handled by GlobalExceptionHandler -> 400, 404, 500
```

19.3 Not Using DTOs

```
// BAD: Exposing entity directly
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userRepository.findById(id).orElseThrow();
    // Response includes: passwordHash, ssn, deletedAt, internalNotes
}

// GOOD: Use DTO
@GetMapping("/users/{id}")
public UserResponse getUser(@PathVariable Long id) {
    return userService.findById(id);
    // Response includes: id, name, email only
}
```

19.4 No Input Validation

```
// BAD: Trust client input blindly
@PostMapping("/users")
public User create(@RequestBody CreateUserRequest request) {
    // name could be empty, email could be "xxx", age could be -5
    return userService.create(request);
}

// GOOD: Validate everything
@PostMapping("/users")
public ResponseEntity<UserResponse> create(@Valid @RequestBody CreateUserRequest
request) {
    // @Valid triggers all validation annotations
    return
ResponseEntity.status(HttpStatus.CREATED).body(userService.create(request));
}
```

19.5 Not Handling Exceptions Globally

```
// BAD: Try-catch in every method
@GetMapping("/{id}")
public ResponseEntity<?> getOrder(@PathVariable Long id) {
    try { ... } catch (OrderNotFoundException e) { ... }
    catch (PaymentFailedException e) { ... }
```

```
    catch (Exception e) { ... }
}

// GOOD: @ControllerAdvice handles ALL exceptions centrally
// Controller code stays clean:
@GetMapping("/{id}")
public OrderResponse getOrder(@PathVariable Long id) {
    return orderService.findById(id);  // Throws exceptions -> handled globally
}
```

19.6 No Pagination

```
// BAD: Return all records
@GetMapping("/orders")
public List<Order> getAll() {
    return orderRepository.findAll();  // 10 million rows -> OutOfMemoryError
}

// GOOD: Always paginate
@GetMapping("/orders")
public Page<OrderResponse> getAll(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
    return orderService.findAll(PageRequest.of(page, size));
}
```

Part 20: Interview Questions & Answers (100+)

Spring Framework Fundamentals (Q1-Q15)

Q1. What is Inversion of Control (IoC)? IoC is a design principle where the control of object creation and dependency management is transferred from the application code to the Spring container. Instead of classes creating their own dependencies, the container provides them.

Q2. What is Dependency Injection? DI is the implementation of IoC. Spring injects the required objects (dependencies) into a class, either through constructor, setter, or field injection.

Q3. Why is constructor injection preferred over field injection? Constructor injection makes dependencies explicit, supports immutability (`final` fields), is easily testable (pass mocks via constructor), and enforces required dependencies at compile time. Field injection hides dependencies and requires reflection for testing.

Q4. What is a Spring Bean? Any object managed by the Spring IoC container. Beans are created, configured, and lifecycle-managed by Spring.

Q5. What are the Bean scopes? `singleton` (default, one instance per container), `prototype` (new instance per injection), `request` (one per HTTP request), `session` (one per HTTP session), `application` (one per ServletContext).

Q6. What is the difference between @Component, @Service, @Repository, and @Controller? All are stereotype annotations for component scanning. `@Component` is generic. `@Service` is for business logic. `@Repository` adds exception translation for data access. `@Controller` is for handling web requests. Functionally equivalent but semantically distinct.

Q7. What is @PostConstruct? A method annotated with `@PostConstruct` runs after the bean is instantiated and dependencies are injected. Used for initialization logic like loading caches.

Q8. What is @PreDestroy? Runs before the bean is removed from the container. Used for cleanup like closing connections.

Q9. What is the BeanFactory vs ApplicationContext? `BeanFactory` is lazy-loading basic container. `ApplicationContext` extends it with eager loading, event publishing, internationalization, and AOP support. Use `ApplicationContext` in production.

Q10. What is @Qualifier used for? Resolves ambiguity when multiple beans of the same type exist: `@Qualifier("mysqlRepo")` specifies which implementation to inject.

Q11. What is @Primary? Marks a bean as the default choice when multiple candidates exist. Used instead of `@Qualifier` when one implementation is clearly preferred.

Q12. What is a BeanPostProcessor? An interface that allows custom modification of beans before and after initialization. Used internally by Spring for proxying, AOP, and annotation processing.

Q13. What happens if there's a circular dependency? Spring throws `BeanCurrentlyInCreationException` for constructor injection. For setter injection, it can resolve circular dependencies by creating partially initialized beans first.

Q14. What is @Lazy? Delays bean creation until it's first requested. Useful for expensive beans that may not always be needed.

Q15. What is @Profile? Activates a bean only for specific profiles: `@Profile("dev")` means this bean is only created when the `dev` profile is active.

Spring Boot (Q16-Q30)

Q16. What is @SpringBootApplication? A convenience annotation combining `@Configuration` (Java config), `@EnableAutoConfiguration` (auto-configure based on classpath), and `@ComponentScan` (scan for beans).

Q17. How does Spring Boot Auto-Configuration work? Spring Boot reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, evaluates `@ConditionalOnClass`, `@ConditionalOnMissingBean`, and other conditions, and creates beans only when conditions are met.

Q18. What is the difference between application.yml and application.properties? Both configure Spring Boot. YAML is hierarchical and more readable. Properties is flat key-value pairs. They are functionally equivalent.

Q19. How do Spring Boot profiles work? Profiles activate different configurations: `application-dev.yml` loads when `spring.profiles.active=dev`. Use for environment-specific settings.

Q20. What is Spring Boot Actuator? Provides production-ready features: `/health` (health checks), `/metrics` (performance), `/info` (app info), `/env` (environment). Essential for monitoring.

Q21. What are Spring Boot Starters? Curated dependency sets. `spring-boot-starter-web` includes Spring MVC, Tomcat, Jackson, and Validation. Eliminates manual dependency management.

Q22. What embedded servers does Spring Boot support? Tomcat (default), Jetty, and Undertow. The application runs as a self-contained JAR with the server inside.

Q23. How to change the default port? `server.port=8081` in `application.yml` or `--server.port=8081` command-line argument.

Q24. What is @ConfigurationProperties? Type-safe binding of properties to a Java class. Maps `app.jwt.secret` to `JwtConfig.secret` field.

Q25. What is Spring Boot DevTools? Development-time features: automatic restart on code change, LiveReload browser refresh, relaxed property binding.

Q26. What is the difference between @Bean and @Component? `@Bean` is used on methods inside `@Configuration` classes to manually create beans. `@Component` is used on classes for automatic component scanning.

Q27. How to create a custom auto-configuration? Create a class with `@AutoConfiguration`, add `@ConditionalOnClass` / `@ConditionalOnMissingBean`, and register in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`.

Q28. What is the spring-boot-maven-plugin for? Packages the application as an executable JAR/WAR with embedded server. Runs with `java -jar app.jar`.

Q29. What is @Value annotation? Injects configuration properties: `@Value("${app.name}")` injects the value of `app.name` from properties.

Q30. What is conditional bean creation? `@ConditionalOnProperty(name="feature.enabled", havingValue="true")` creates a bean only when a specific property is set.

Spring MVC (Q31-Q50)

Q31. What is DispatcherServlet? The front controller that receives ALL incoming HTTP requests and dispatches them to the correct handler (controller method). It's auto-configured by Spring Boot.

Q32. What is HandlerMapping? Maps incoming request URLs to controller methods. `RequestMappingHandlerMapping` handles `@RequestMapping` annotations.

Q33. What is the difference between @Controller and @RestController? `@Controller` returns view names (for server-side rendering). `@RestController` = `@Controller` + `@ResponseBody`, returns data directly as JSON.

Q34. What is @RequestMapping? Maps HTTP requests to handler methods. Can specify URL path, HTTP method, content type, etc.

Q35. What is the difference between @GetMapping and @RequestMapping(method=GET)? `@GetMapping` is a shortcut for `@RequestMapping(method = RequestMethod.GET)`. More concise and readable.

Q36. What is @PathVariable? Extracts values from URL path segments: `/orders/{id}` -> `@PathVariable Long id` gets `123` from `/orders/123`.

Q37. What is @RequestParam? Extracts query string parameters: `/orders?status=PENDING` -> `@RequestParam String status` gets `"PENDING"`.

Q38. What is the difference between @PathVariable and @RequestParam? `@PathVariable` extracts from URL path (identity: `/orders/123`). `@RequestParam` extracts from query string (filtering: `?status=PENDING`).

Q39. What is @RequestBody? Deserializes the HTTP request body (typically JSON) into a Java object using Jackson `ObjectMapper`.

Q40. What is @ResponseBody? Serializes the return value of a controller method to JSON and writes it to the HTTP response body. Included automatically in `@RestController`.

Q41. What is ResponseEntity? A class that represents the full HTTP response: status code, headers, and body. Gives complete control over the response.

Q42. What is @ModelAttribute? Binds form data to a Java object. Used in server-side rendered applications with Thymeleaf.

Q43. What are HandlerInterceptors? Components that execute before and after controller methods. Used for logging, authentication, request timing.

Q44. What is WebMvcConfigurer? An interface to customize Spring MVC configuration: CORS, interceptors, formatters, view resolvers.

Q45. How does content negotiation work? Spring checks `Accept` header and `Content-Type` to determine request/response format. Returns JSON for `Accept: application/json`, XML for `Accept: application/xml`.

Q46. What is HttpMessageConverter? Converts Java objects to/from HTTP messages. `MappingJackson2HttpMessageConverter` handles JSON. `Jaxb2RootElementHttpMessageConverter` handles XML.

Q47. What is @CrossOrigin? Configures CORS (Cross-Origin Resource Sharing) for specific endpoints. Allows requests from different domains.

Q48. What is Filter vs HandlerInterceptor? `Filter` operates at Servlet level (before DispatcherServlet). `HandlerInterceptor` operates at Spring MVC level (after DispatcherServlet, before controller).

Q49. What is the difference between forward and redirect? Forward: server-side, same request, URL doesn't change. Redirect: sends 302 to client, new request, URL changes.

Q50. What is @SessionAttribute? Stores/retrieves attributes from the HTTP session. Used for multi-step forms or session-scoped data.

REST API Design & Implementation (Q51-Q80)

Q51. What is REST? Representational State Transfer -- an architectural style for distributed systems using HTTP. Based on resources, standard HTTP methods, and stateless communication.

Q52. What are the REST architectural constraints? Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, Code on Demand (optional).

Q53. What is a resource in REST? Any entity that can be identified by a URL. `/api/orders/123` identifies a specific order resource.

Q54. What HTTP method should be used for creating resources? POST. Returns 201 Created with a `Location` header pointing to the new resource.

Q55. What is idempotency? A request is idempotent if making it multiple times has the same effect. GET, PUT, DELETE are idempotent. POST is NOT.

Q56. What is HATEOAS? Hypermedia as the Engine of Application State. API responses include links to related actions and resources, making the API self-discoverable.

Q57. What is the difference between PUT and PATCH? PUT replaces the entire resource (all fields required). PATCH modifies specific fields only (partial update).

Q58. What HTTP status code for validation error? 400 Bad Request for syntactic/format errors. 422 Unprocessable Entity for semantically invalid but well-formed requests.

Q59. What HTTP status code for authentication failure? 401 Unauthorized (not authenticated). 403 Forbidden (authenticated but not authorized).

Q60. Why should entities not be exposed in API responses? Entities may contain sensitive fields (passwords, SSNs), internal audit fields, and JPA proxies that cause serialization errors. DTOs provide a controlled API contract.

Q61. What is the DTO pattern? Creating separate Java objects for API requests (input) and responses (output) that differ from database entities.

Q62. What is the Builder pattern for DTOs? Using `@Builder` (Lombok) to create DTOs fluently: `OrderResponse.builder().id(1L).status("CONFIRMED").build()`.

Q63. What is MapStruct? A compile-time code generator for mapping between DTOs and entities. Generates type-safe mapping code, avoiding runtime reflection.

Q64. What is @Valid and how does it work? Triggers JSR 380 Bean Validation on the annotated parameter. Validates all constraint annotations (`@NotNull` , `@Size` , etc.) and throws `MethodArgumentNotValidException` on failure.

Q65. What is a custom validator? A class implementing `ConstraintValidator<A, T>` paired with a custom annotation. Used for validation rules not covered by standard annotations.

Q66. What is @ControllerAdvice? A global exception handler that applies to all controllers. Methods annotated with `@ExceptionHandler` handle specific exception types centrally.

Q67. What is the difference between @ControllerAdvice and @RestControllerAdvice? `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`. Returns JSON error responses directly.

Q68. How to handle 404 Not Found in REST APIs? Throw a `ResourceNotFoundException` from service layer, handle it in `@ControllerAdvice` with `@ExceptionHandler`, return 404 status with error body.

Q69. What is API versioning? Supporting multiple API versions simultaneously. Strategies: URI path (`/v1/orders`), custom header (`X-API-Version: 1`), media type (`application/vnd.company.v1+json`).

Q70. Which API versioning strategy is best? URI path versioning for public APIs (simplest, most common). Header versioning for internal APIs (cleaner URLs).

Q71. What is OpenAPI / Swagger? A specification for describing REST APIs. SpringDoc auto-generates interactive documentation from controller annotations.

Q72. What is pagination and why is it important? Returning data in fixed-size pages instead of all at once. Prevents out-of-memory errors, reduces response time, and improves UX.

Q73. How to implement pagination in Spring Boot? Accept `Pageable` in repository methods: `findAll(Pageable pageable)`. Use `PageRequest.of(page, size, sort)` to create pageable objects.

Q74. What is Spring Data JPA Specification? Dynamic query building using the Criteria API. `Specification<Order>` enables composable, type-safe queries for complex search filters.

Q75. What is the N+1 query problem? Loading a list of entities (1 query) and then loading each entity's relationships one by one (N queries). Fix with `JOIN FETCH` or `@EntityGraph`.

Q76. What is @Transactional in service layer? Wraps the method in a database transaction. On exception, all changes are rolled back. Ensures data consistency.

Q77. What is the difference between @Transactional(readOnly=true) and regular? `readOnly=true` optimizes reads -- Hibernate skips dirty checking, databases may choose read replicas. Use for GET operations.

Q78. What is CORS and how to configure it? Cross-Origin Resource Sharing -- allows web browsers to make requests to different domains. Configure with `@CrossOrigin` or `WebMvcConfigurer.addCorsMappings()`.

Q79. What is multipart file upload? Handling file uploads via `MultipartFile` parameter with `@RequestParam("file")`. Configure `spring.servlet.multipart.max-file-size`.

Q80. What is content compression in REST APIs? GZIP compression of response bodies. Reduces payload size by 60-90%. Configure `server.compression.enabled=true`.

Security & Performance (Q81-Q105)

Q81. What is JWT? JSON Web Token -- a compact, URL-safe token for authentication. Contains header, payload (claims like user ID, roles), and signature.

Q82. What is the JWT authentication flow? Client sends credentials -> Server validates -> Server creates JWT -> Client includes JWT in `Authorization: Bearer <token>` header -> Server validates token on each request.

Q83. What is Spring Security Filter Chain? A chain of servlet filters that processes every HTTP request. Includes `SecurityContextPersistenceFilter`, `UsernamePasswordAuthenticationFilter`, `ExceptionTranslationFilter`, etc.

Q84. What is the difference between authentication and authorization? Authentication verifies identity (who are you?). Authorization verifies permissions (what can you do?).

Q85. What is @PreAuthorize? Method-level security: `@PreAuthorize("hasRole('ADMIN')")` restricts method access to users with ADMIN role.

Q86. What is CSRF and why disable it for APIs? Cross-Site Request Forgery -- attacks that trick browsers into making unauthorized requests. Disabled for stateless APIs because JWT tokens provide built-in CSRF protection.

Q87. What is OAuth2? An authorization framework for delegated access. Allows users to grant third-party applications access without sharing passwords.

Q88. What is the difference between OAuth2 and JWT? OAuth2 is a protocol/framework for authorization flows. JWT is a token format. OAuth2 often USES JWT as the token format.

Q89. What is rate limiting? Restricting the number of API requests a client can make in a time window. Prevents abuse and DDoS attacks. Returns 429 Too Many Requests.

Q90. What is API caching? Storing frequently accessed data to avoid repeated database queries. Use `@Cacheable` with Redis or in-memory cache. Improves response time significantly.

Q91. What is @Cacheable vs @CacheEvict? `@Cacheable` populates the cache on first call, returns cached value on subsequent calls. `@CacheEvict` removes entries from the cache (on update/delete).

Q92. What is ETag caching? HTTP-level caching where the server returns an `ETag` header (hash of response). Client sends `If-None-Match` header -- if data hasn't changed, server returns 304 Not Modified.

Q93. What is connection pooling? Reusing database connections instead of creating new ones for each request. HikariCP is Spring Boot's default pool.

Q94. What is @Async in Spring? Executes a method in a separate thread: `@Async public void sendEmail(...)`. Requires `@EnableAsync` on a configuration class.

Q95. What is WebSocket vs REST? REST is request-response (client pulls). WebSocket is bidirectional persistent connection (server pushes). Use WebSocket for real-time features like chat.

Q96. What is Server-Sent Events (SSE)? One-way server-to-client streaming over HTTP. Simpler than WebSocket for scenarios like live notifications.

Q97. What is API Gateway? A single entry point for all microservice APIs. Handles routing, authentication, rate limiting, and load balancing. Examples: Spring Cloud Gateway, Kong.

Q98. What is the difference between synchronous and asynchronous APIs? Synchronous: client waits for response. Asynchronous: server accepts request, returns 202 Accepted, processes in background, client polls for result.

Q99. What is request-response logging? Logging every HTTP request and response for debugging and auditing. Implement via `Filter` or `HandlerInterceptor`. Include request ID for tracing.

Q100. What is API idempotency? Designing APIs so that retrying the same request produces the same result. Essential for payments -- use idempotency keys to prevent double charges.

Q101. How to handle file downloads in REST APIs? Return `byte[]` or `InputStreamResource` with `Content-Disposition: attachment` header and appropriate `Content-Type`.

Q102. What is Spring HATEOAS? A Spring project for building HATEOAS-compliant REST APIs. Adds `_links` with navigable URLs to API responses.

Q103. What is the difference between monolith and microservices? Monolith: single deployable unit with all features. Microservices: multiple independent services communicating via APIs. Microservices offer independent scaling and deployment.

Q104. What is RestTemplate vs WebClient? `RestTemplate` is synchronous (blocking). `WebClient` is reactive (non-blocking, supports async). Use `WebClient` for new projects.

Q105. What is the most important REST API design principle? Consistency. Consistent URL patterns, response formats, status codes, and error handling across all endpoints. An API should be predictable.

Part 21: Hands-on Practice Exercises

Exercise 1: Build a Complete CRUD REST API

Scenario: Build a Product Management API.

Endpoints:

- `GET /api/v1/products` -- list products with pagination
- `GET /api/v1/products/{id}` -- get by ID
- `POST /api/v1/products` -- create product
- `PUT /api/v1/products/{id}` -- update product
- `DELETE /api/v1/products/{id}` -- delete product

Requirements:

1. Use `@RestController`, proper HTTP methods
2. Return correct status codes (200, 201, 204, 404)
3. Use `ResponseEntity` for all responses
4. Include pagination with `Pageable`

Exercise 2: Add Validation

Extend Exercise 1 to add:

1. `@NotBlank` on product name
2. `@Positive` on price
3. `@Size(min=10, max=500)` on description
4. Return structured validation error response (field name + error message)
5. Handle `MethodArgumentNotValidException` globally

Exercise 3: Implement Global Exception Handling

Create a `@RestControllerAdvice` that handles:

1. `ResourceNotFoundException` -> 404
2. `DuplicateResourceException` -> 409
3. `MethodArgumentNotValidException` -> 400
4. Generic `Exception` -> 500 (log error, return safe message)

All error responses must follow this structure:

```
{
  "timestamp": "2024-01-15T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Product not found with id: 99",
  "path": "/api/v1/products/99"
}
```

Exercise 4: Implement DTO Layer

Refactor Exercise 1:

1. Create `CreateProductRequest` (name, price, description, categoryId)
2. Create `UpdateProductRequest` (name, price, description)
3. Create `ProductResponse` (id, name, price, categoryName, createdAt)
4. Write manual mapper methods or use MapStruct
5. Ensure entity is never exposed in API responses

Exercise 5: Add Pagination and Sorting

Extend the `GET /api/v1/products` endpoint:

1. Accept `page`, `size`, `sortBy`, `direction` query params
2. Return paginated response with metadata (totalElements, totalPages, etc.)
3. Default: page=0, size=20, sortBy=createdAt, direction=desc
4. Add filtering by category: `/api/v1/products?category=ELECTRONICS`

Exercise 6: API Versioning and Documentation

Create v2 of the Products API:

1. v1 returns: `{ id, name, price }`
2. v2 returns: `{ id, name, price, formattedPrice, inStock, tags }`
3. Both versions work simultaneously
4. Add Swagger annotations for v2 endpoints
5. Configure OpenAPI with title, description, version, and JWT security scheme
6. Verify at: `http://localhost:8080/swagger-ui.html`

End of Guide

> *This guide covered Spring Boot MVC and REST API development from framework fundamentals to production-grade architecture. The key principles:* ***use proper HTTP semantics****,* ***separate concerns with layers****,* ***validate all input****,* ***handle exceptions globally****, and* ***never expose entities directly****. Build APIs that are consistent, well-documented, and secure.*